Day 21: Kubernetes Security, RBAC & Troubleshooting

COVER SLIDE / OVERVIEW

```
ubuntu@master-node:~$ kubectl

# RBAC Authorization, Pod Security Standards & Debugging
$ kubectl create namespace secure
$ kubectl apply -f rbac.yaml
$ kubectl label namespace secure pod-
security.kubernetes.io/enforce=restricted
$ kubectl apply -f secure-pod.yaml
Status: Least-Privilege RBAC & Restricted Pod Security Verified
```

MISSION ROADMAP (01 TO 08)

- ✓ **01: RBAC Role, ServiceAccount & Can-I Check (Task A)**
- ✓ **02: Namespace Restricted Pod Security Enforcement (Task B)**
- ✓ **03: Hardened Non-Root Pod Deployment (Task B)**
- ✓ **04: Root Pod Policy Violation Rejection (Task B)**
- ✓ **05: Three Broken Deployments Setup & Diagnostics (Task C)**
- ✓ **06: Fixing ImagePullBackOff (broken1)**
- ✓ **07: Fixing CrashLoopBackOff (broken2)**
- ✓ **08: Fixing Pending Resource Limits (broken3) & Cleanup**

01 / Core Concepts: RBAC, Pod Security & Debugging Workflow

Security & Authorization Mechanisms

- **ServiceAccount:** Identity used by pods, applications, or automation tools to authenticate to the API server.
- **Role & RoleBinding:** Namespaced permissions and bindings. Deny-by-default; grant only required verbs.
- **Pod Security Standards:** Privileged, Baseline, and Restricted profiles. Restricted is the strongest profile.
- **Hardened SecurityContext:** Enforces `runAsNonRoot: true`, `readOnlyRootFilesystem: true`, and capability drops.
- **Debug Order:** `get → describe → logs → events → exec`. Always inspect status and Events first.

Essential kubectl Commands for Day 21

- `kubectl auth can-i ... --as-system:...` — Verify effective RBAC permissions.
- `kubectl label namespace secure ...` — Enforce Pod Security Standards on a namespace.
- `kubectl describe pod <pod-name> -n lab` — Inspect scheduling failures and probe events.
- `kubectl logs <pod-name> -n lab --previous` — Retrieve crash logs from exited containers.
- `kubectl set image deployment/...` — Update container image tags to resolve pull errors.
- `kubectl delete namespace secure lab` — Clean up lab resources after verification.

02 / RBAC Setup & Permission Verification (Task A)

Task A: Created namespace `secure`, ServiceAccount `viewer`, namespace `lab`, and applied `rbac.yaml` granting read-only pod access. Verified with `kubectl auth can-i`.

```
ubuntu@master-node:~$ kubectl create namespace secure
namespace/secure created
ubuntu@master-node:~$ kubectl create serviceaccount viewer -n secure
serviceaccount/viewer created
ubuntu@master-node:~$ kubectl create namespace lab
namespace/lab created
ubuntu@master-node:~$ nano rbac.yaml
ubuntu@master-node:~$ cat rbac.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: secure
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: viewer-pods
  namespace: secure
subjects:
- kind: ServiceAccount
  name: viewer
  namespace: secure
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
ubuntu@master-node:~$ kubectl apply -f rbac.yaml
role.rbac.authorization.k8s.io/pod-reader created
rolebinding.rbac.authorization.k8s.io/viewer-pods created
ubuntu@master-node:~$ kubectl auth can-i get pods -n secure --as-system:serviceaccount:secure:viewer
error: unknown flag: --as-system:serviceaccount:secure:viewer
See 'kubectl auth can-i --help' for usage.
ubuntu@master-node:~$ ^C
ubuntu@master-node:~$ kubectl auth can-i get pods -n secure --as=system:serviceaccount:secure:viewer
yes
ubuntu@master-node:~$ kubectl auth can-i delete pods -n secure --as=system:serviceaccount:secure:viewer
no
```

Task A Execution Highlights

- Created namespace `secure` and ServiceAccount `viewer`.
- Authored `rbac.yaml` defining a Role (`pod-reader`) with verbs `get`, `list`, `watch` on resources `pods` and `pods/log`.
- Bound role via RoleBinding (`viewer-pods`).
- Verified permissions: `kubectl auth can-i get pods` returned **yes**, and `delete pods` returned **no**.

03 / Restricted Pod Security Enforcement & Secure Pod (Task B)

Task B: Labeled namespace `secure` with restricted enforcement and deployed a fully hardened `secure-pod.yaml` running as non-root with read-only root filesystem.

```
ubuntu@master-node:~$ kubectl label namespace secure pod-security.kubernetes.io/enforce=restricted
namespace/secure labeled
ubuntu@master-node:~$ nano secure-pod.yaml
ubuntu@master-node:~$ cat secure-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
  namespace: secure
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: app
    image: busybox:1.36
    command: ["sleep", "3600"]
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
ubuntu@master-node:~$ kubectl apply -f secure-pod.yaml
pod/secure-pod created
ubuntu@master-node:~$ kubectl get pod secure-pod -n secure
NAME         READY   STATUS    RESTARTS   AGE
secure-pod   1/1     Running   0          31s
ubuntu@master-node:~$ kubectl describe pod secure-pod -n secure
Name:             secure-pod
Namespace:        secure
Priority:         0
Service Account:  default
```

Hardened Pod Specifications

- Enforced restricted policy: `kubectl label namespace secure pod-security.kubernetes.io/enforce=restricted`.
- Configured `runAsNonRoot: true` and `runAsUser: 10001`.
- Applied `seccompProfile: RuntimeDefault` and dropped all capabilities.
- Verified `secure-pod` successfully reaches `Running (1/1)` status.

04 / Root Pod Policy Violation Rejection (Task B)

Task B (Cont.): Attempted to deploy a root-privileged pod to test policy enforcement. The admission controller successfully rejected the pod for violating restricted Pod Security Standards.

```
ubuntu@master-node:~$ kubectl run root-pod --image=nginx:1.27 -n secure
Error from server (Forbidden): pods "root-pod" is forbidden: violates PodSecurity "restricted:latest":
allowPrivilegeEscalation != false (container "root-pod" must set securityContext.allowPrivilegeEscalati
on=false), unrestricted capabilities (container "root-pod" must set securityContext.capabilities.drop=[
"ALL"]), runAsNonRoot != true (pod or container "root-pod" must set securityContext.runAsNonRoot=true),
 seccompProfile (pod or container "root-pod" must set securityContext.seccompProfile.type to "RuntimeDe
fault" or "Localhost")
```

Policy Enforcement Evidence

- Executed `kubectl run root-pod --image=nginx:1.27 -n secure`.
- **Rejection Error:** Forbidden due to missing privilege escalation settings, unrestricted capabilities, missing non-root execution, and default seccomp profile.
- Proves that the restricted policy actively safeguards namespace workloads without weakening security.

05 / Three Broken Deployments Creation & Initial Inspection (Task C)

Task C: Created namespace `lab` and deployed three intentionally broken workloads (`broken1`, `broken2`, `broken3`) to practice systematic root-cause debugging.

```
ubuntu@master-node:~$ kubectl create deploy broken1 --image=nginx:1.99.99 -n lab
deployment.apps/broken1 created
ubuntu@master-node:~$ kubectl create deploy broken2 --image=busybox:1.36 -n lab -- sh -c "exit 1"
deployment.apps/broken2 created
ubuntu@master-node:~$ kubectl create deploy broken3 --image=nginx:1.27 -n lab
deployment.apps/broken3 created
ubuntu@master-node:~$ kubectl set resources deploy broken3 -n lab --requests=memory=64Gi
deployment.apps/broken3 resource requirements updated
ubuntu@master-node:~$ kubectl get deployments -n lab
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
broken1   0/1     1            0           96s
broken2   0/1     1            0           87s
broken3   1/1     1            1           78s
ubuntu@master-node:~$ kubectl get pods -n lab
NAME                       READY   STATUS             RESTARTS      AGE
broken1-5b97886888-jxb7f   0/1     ErrImagePull       0             113s
broken2-745966599-lvh7w    0/1     CrashLoopBackOff   4 (15s ago)   104s
broken3-6cc8f686f5-vfch6   1/1     Running            0             95s
broken3-74f5b74f8d-ff84t   0/1     Pending            0             87s
ubuntu@master-node:~$ kubectl describe pod broken1-5b97886888-jxb7f -n lab
```

Initial Symptoms Observed

- `broken1` (nginx:1.99.99): **ErrImagePull** due to invalid image tag.
- `broken2` (busybox running `exit 1`): **CrashLoopBackOff** due to immediate exit.
- `broken3` (nginx requesting 64Gi): **Pending** because kind node lacks 64Gi RAM.

06 / Repairing broken1 (Image Tag) & broken3 (Resources)

Task C (Cont.): Fixed `broken1` by updating to a valid image tag (`nginx:1.27`) and repaired `broken3` by reducing memory request from 64Gi to 64Mi.

```
ubuntu@master-node:~$ kubectl set image deployment/broken1 -n lab nginx=nginx:1.27
deployment.apps/broken1 image updated
ubuntu@master-node:~$ kubectl get pod broken1-5b97886888-jxb7f -n lab
Error from server (NotFound): pods "broken1-5b97886888-jxb7f" not found
ubuntu@master-node:~$ kubectl get deployments -n lab
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
broken1   1/1     1            1           19m
broken2   0/1     1            0           19m
broken3   1/1     1            1           19m
ubuntu@master-node:~$ kubectl get pods -n lab
NAME                       READY   STATUS             RESTARTS        AGE
broken1-676db6584f-xkkl5   1/1     Running            0               5m36s
broken2-745966599-lvh7w    0/1     CrashLoopBackOff   8 (3m25s ago)   19m
broken3-6cc8f686f5-vfch6   1/1     Running            0               19m
broken3-74f5b74f8d-ff84t   0/1     Pending            0               19m
```

```
ubuntu@master-node:~$ kubectl set resources deployment/broken3 -n lab --requests=memory=64Mi
deployment.apps/broken3 resource requirements updated
ubuntu@master-node:~$ kubectl get pods -n lab
NAME                       READY   STATUS             RESTARTS        AGE
broken1-676db6584f-xkkl5   1/1     Running            0               11m
broken2-745966599-lvh7w    0/1     CrashLoopBackOff   9 (4m28s ago)   25m
broken3-75dfd4dc58-c4fzs   1/1     Running            0               5s
```

07 / Repairing broken2 (Command Fix) & Full Cluster Verification

Task C (Cont.): Patched `broken2` container command to a long-running process (`sleep 3600`) and verified all three deployments and pods reach `Running (1/1)`.

```
ubuntu@master-node:~$ kubectl patch deployment broken2 -n lab --type=strategic -p '{"spec":{"template":{"spec":{"containers":[{"name":"busybox","command":["sleep"],"args":["3600"]}]}}}}'
deployment.apps/broken2 patched
ubuntu@master-node:~$ kubectl delete pod -l app=broken2 -n lab
pod "broken2-b86dc56b-gnvkn" deleted
ubuntu@master-node:~$ kubectl get pods -n lab
NAME                       READY   STATUS    RESTARTS   AGE
broken1-676db6584f-xkkl5   1/1     Running   0          24m
broken2-b86dc56b-2g599     1/1     Running   0          44s
broken3-75dfd4dc58-c4fzs   1/1     Running   0          13m
ubuntu@master-node:~$
```

Repair Verification Summary

- `broken1-676db6584f-xkkl5`: Running (0 restarts).
- `broken2-b86dc56b-2g599`: Running (0 restarts) after sleep patch.
- `broken3-75dfd4dc58-c4fzs`: Running (0 restarts) with realistic 64Mi memory request.

08 / Troubleshooting & Debug Notes Matrix

Deployment	Symptom / Evidence	Root Cause	Fix / Final Status
broken1	ErrImagePull / ImagePullBackOff	Invalid image tag (`nginx:1.99.99`)	Set valid tag (`nginx:1.27`) via `kubectl set image`. Status: **Running (1/1)**.
broken2	CrashLoopBackOff (Exit code 1)	Container command exited immediately (`sh -c "exit 1"`)	Patched command to long-running process (`sleep 3600`). Status: **Running (1/1)**.
broken3	Pending (Scheduling failure)	Excessive memory request (`64Gi`) exceeds node capacity	Reduced memory request to `64Mi` via `kubectl set resources`. Status: **Running (1/1)**.

09 / Lab Cleanup & Repository Deliverables

Successfully completed lab objectives, verified RBAC and Pod Security Standards, repaired broken deployments, and cleaned up lab namespaces.

```
ubuntu@master-node:~$ kubectl delete namespace secure lab
namespace "secure" deleted
namespace "lab" deleted
```

GitHub Repository Structure

- **Directory:** `day21-security-debug/`
- **Files:** `rbac.yaml`, `secure-pod.yaml`, `README.md`
- **Folder:** `screenshot/` containing task execution evidence.
- **Cleanup Command:** `kubectl delete namespace secure lab`.

10 / Interview Readiness & Architecture Q&A

1. What is RBAC and how do you verify it?

- Controls who can perform actions on resources within a scope. Verified using `kubectl auth can-i`.

2. Why avoid running containers as root?

- Non-root execution significantly limits the blast radius of a compromised or vulnerable application.

3. What are Pod Security Standards?

- Profiles named Privileged, Baseline, and Restricted that enforce security best practices at the namespace level.

4. What is the standard debugging order?

- `get → describe → logs → events → exec`. Always inspect status and Events before modifying configurations.